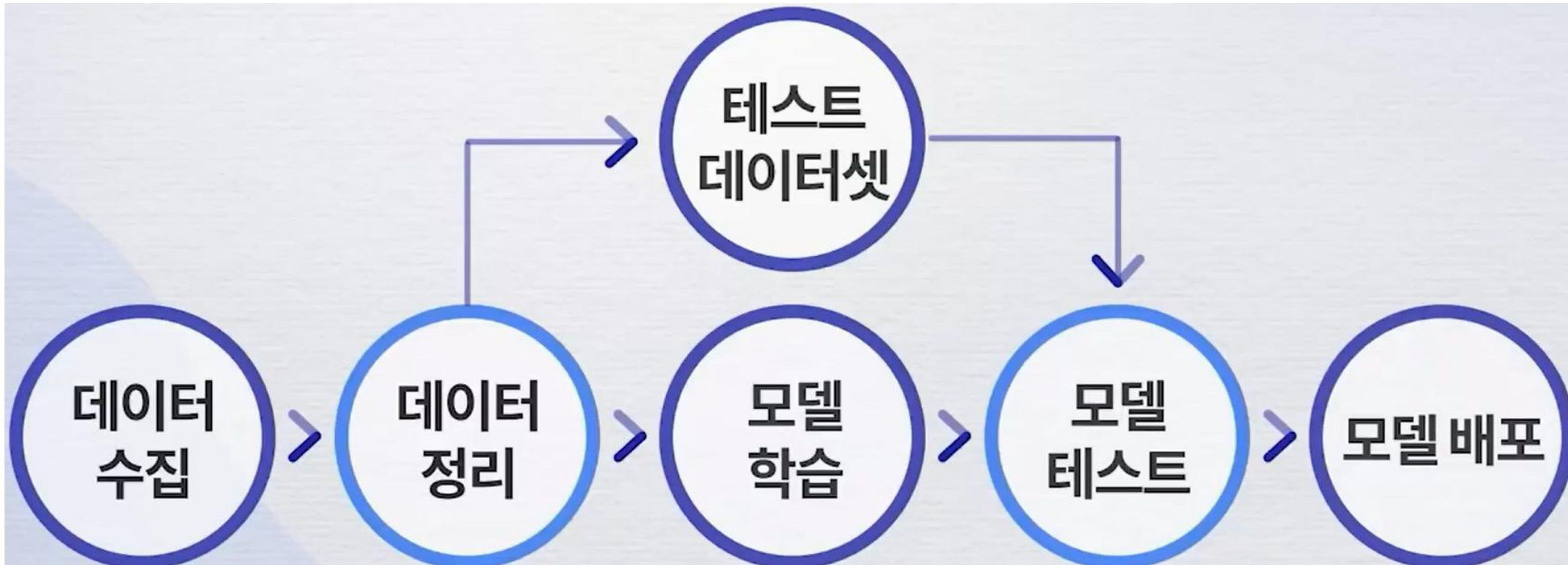


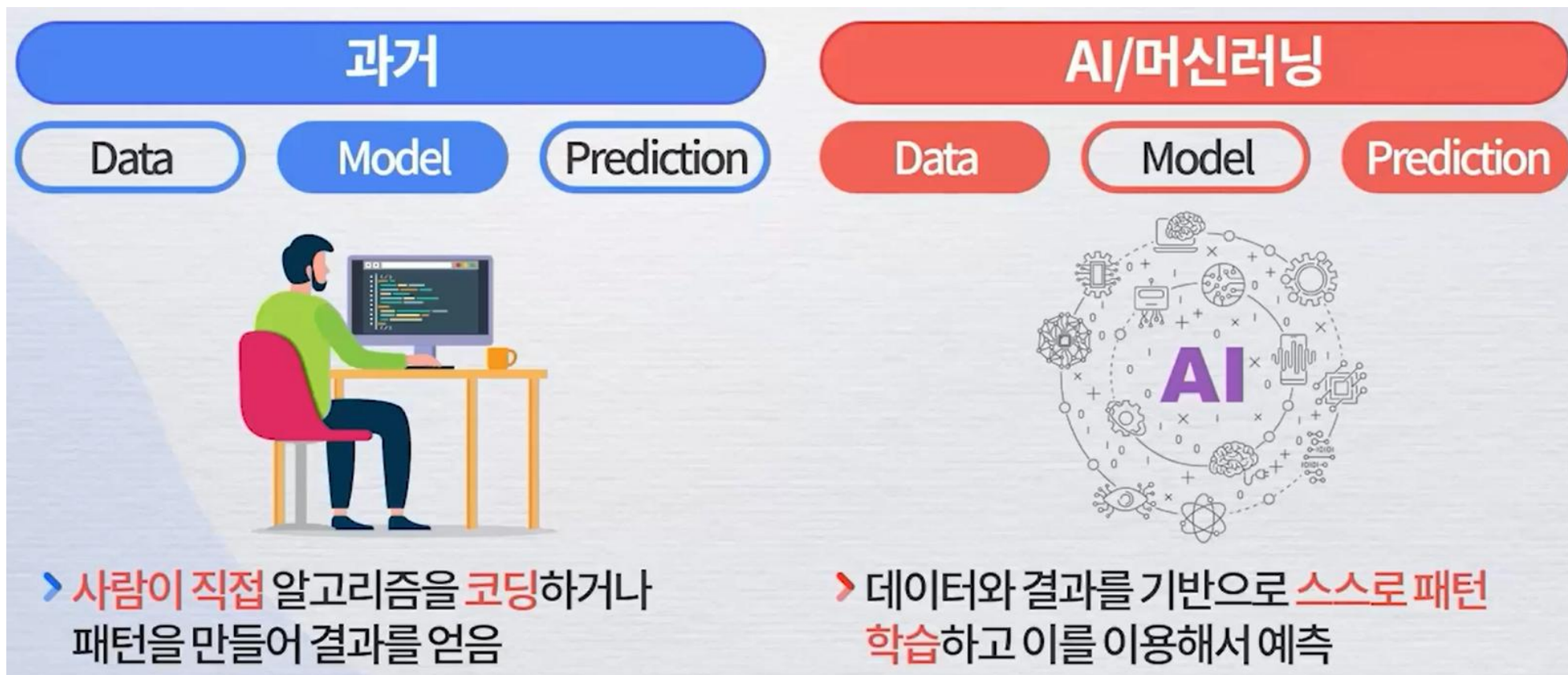
머신러닝

- 머신러닝 기본 개념
- 머신러닝 기술원리
- 머신러닝 주요 알고리즘
- 파이썬 실습

머신러닝 프로세스



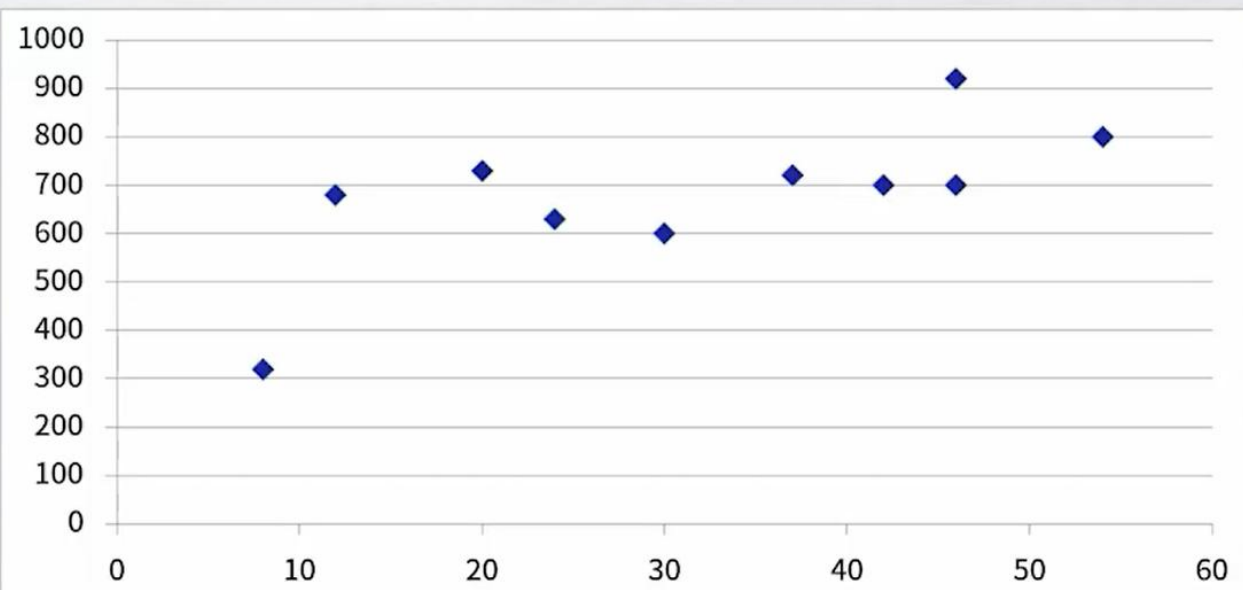
머신러닝 개념(2/10)



Linear Regression

가설 : “공부를 많이 하면 공부를 잘할 것이다.”

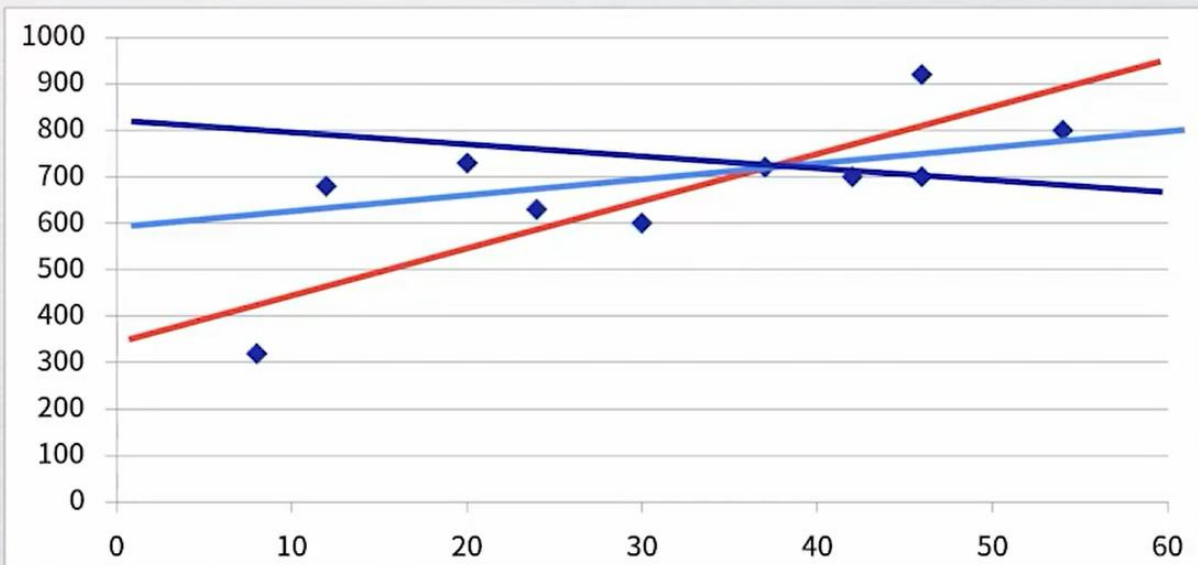
학습시간(시간)	토익성적(점)
54	800
8	320
30	600
24	630
46	700
12	680
20	730
37	720
42	700
46	920



Linear Regression

우리의 목표는?

학습시간(시간)	토익성적(점)
54	800
8	320
30	600
24	630
46	700
12	680
20	730
37	720
42	700
46	920



$$y = ax + b$$

$$y = -0.2x + 850$$

$$y = 0.2x + 600$$

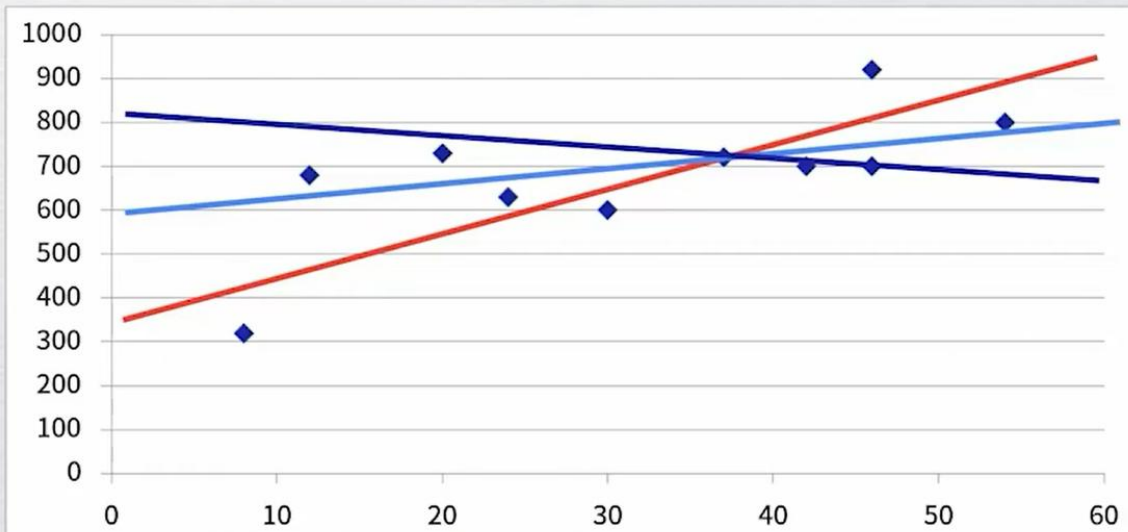
$$y = 0.5x + 350$$

함수, 모델, 알고리즘

Linear Regression

가장 잘 맞는 직선을 구한다는 의미는?

학습시간(시간)	토익성적(점)
54	800
8	320
30	600
24	630
46	700
12	680
20	730
37	720
42	700
46	920



$$y = wx + b$$

단순화

$$y = wx$$

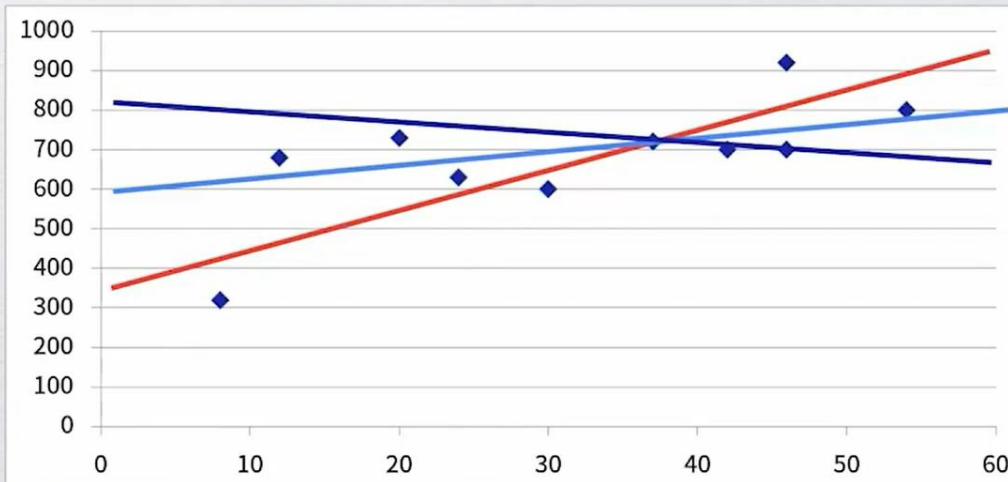
y : 토익성적
x : 학습시간
w : 가중치

많은 후보 w 중에서
점들의 분포패턴에 가장 잘 맞는
w를 찾아야 함
-> 빨간색 직선에 해당하는 w 찾기

Linear Regression

컴퓨터는 W 가중치가 최적인지 어떻게 알 수 있을까?

학습시간(시간)	토익성적(점)
54	800
8	320
30	600
24	630
46	700
12	680
20	730
37	720
42	700
46	920

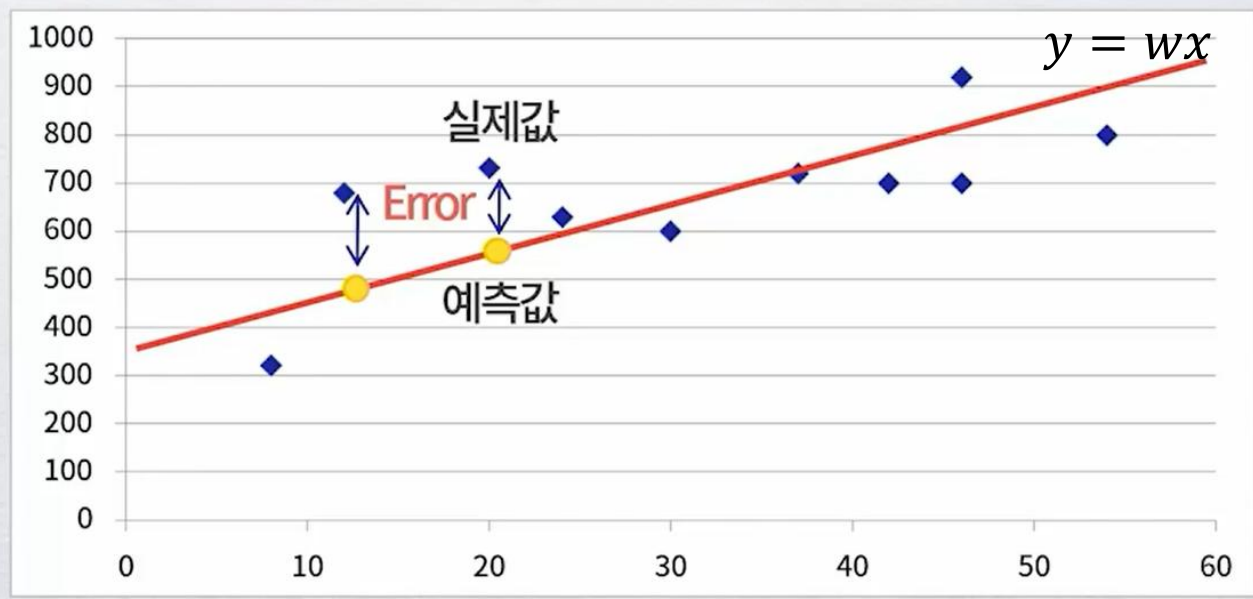


많은 후보 w 중에서
점들의 분포패턴에 가장 잘 맞는
 w 를 찾아야 함
→ 빨간색 직선에 해당하는 w 찾음

비용함수(**Cost Function**)를 최소로 하는
 w 를 구함

MSE(Mean Squared Error, 평균제곱오차)
를 최소화하는 w 를 찾음

Linear Regression



예측값 - 실제값 = Error, cost

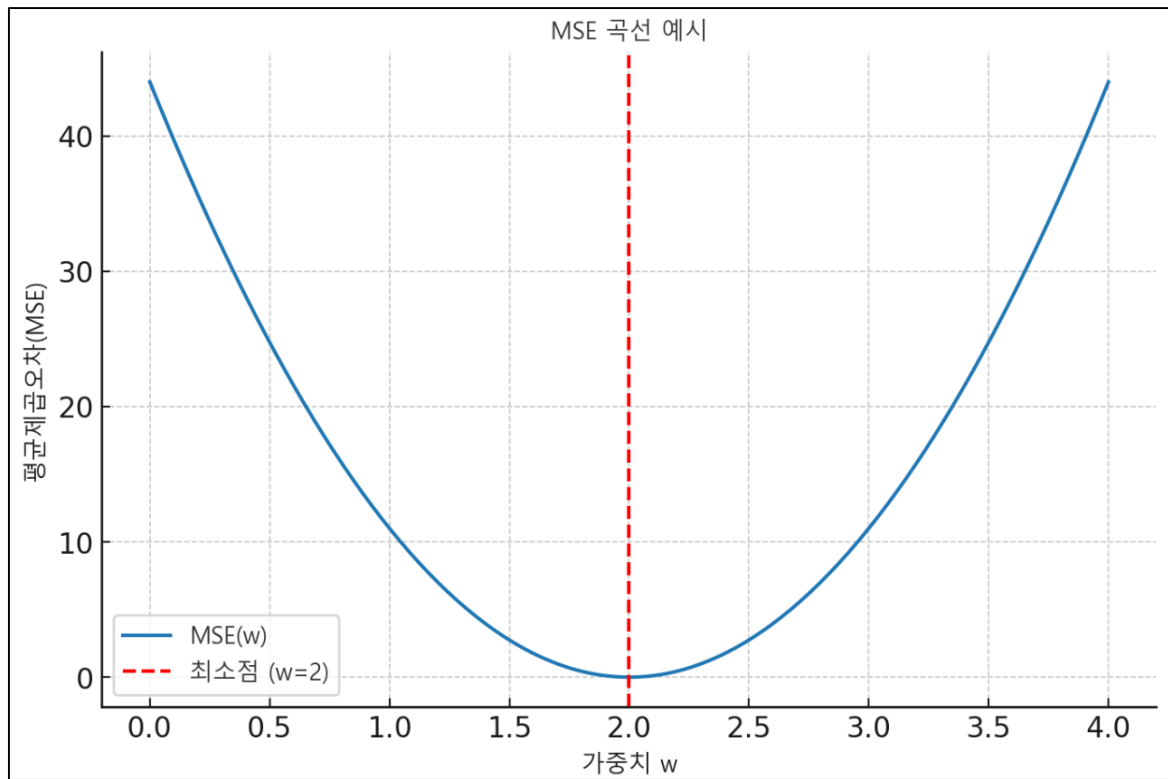
예측값과 실제값의 차이를 제곱하여 모두 더한 뒤, 데이터 개수로 나눈 값이 MSE
(**MSE**는 Linear Regression의 **비용함수**)

$$MSE = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

- n : 데이터의 개수
- y_i : 실제값 (Actual value)
- $\hat{y}_i$: 예측값 (Predicted value)
- $(\hat{y}_i - y_i)^2$: 예측값과 실제값의 차이를 제곱한 값
- 평균을 내기 위해 $\frac{1}{n}$ 을 곱함

머신러닝 개념(8/10)

MSE 비용함수의 2차원 그래프 모양(



$$\text{MSE}(w) = \frac{1}{n} \sum_{i=1}^n (wx_i - y_i)^2$$

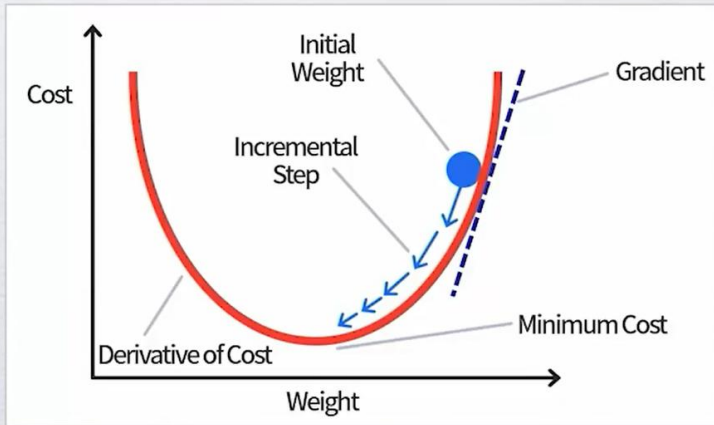
$$\text{MSE}(w) = \frac{1}{n} \sum_{i=1}^n (w^2 x_i^2 - 2wx_i y_i + y_i^2)$$

- w^2 계수가 항상 양수이므로 아래로 볼록한 U자형 곡선이 됨
- 최저점은 MSE가 최소가 되는 w 값(즉, 최적 파라미터)임
- 2차원 좌표평면에서는 x축에 w , y축에 MSE 값을 두면 그려짐

Cost 가장 낮아지는 수준까지 진행하면서 가중치 업데이트 진행

Gradient Descent Algorithm

그래프상에서 비용이 제일 낮은 아래부분(기울기 0)을 찾으면 된다!



- 최적의 w 를 찾기 위하여 **경사하강법**을 이용
- 임의의 w 로부터 시작하여 w 를 조금씩 감소시키면서 가장 작은 기울기를 나타내는 w 를 찾음
- 비용함수를 미분하면서 w 를 찾음

$$W := W - \alpha \frac{\partial}{\partial W} \text{cost}(W)$$

학습률
(w 를 감소시키는 정도)

$$\frac{1}{n} \sum_{i=1}^n (w^2 x_i^2 - 2w x_i y_i + y_i^2)$$

모델 학습이란?

목표: 최적 직선 구하기



직선별 손실함수 구하기



손실함수 최소값 구하기



Gradient Descent Algorithm 이용

$$y = wx$$

$$\text{MSE}(w) = \frac{1}{n} \sum_{i=1}^n (w^2 x_i^2 - 2wx_i y_i + y_i^2)$$

$$W := W - \alpha \frac{\partial}{\partial W} \text{cost}(W)$$

데이터 확보

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

잘 정리된 데이터 확보가 중요!

지도학습 (Supervised Learning)

- ▶ 정답을 알려주면서 진행하는 학습
- ▶ 데이터와 레이블(정답)이 함께 제공됨

레이블(Label) = 정답, 실제값, 타겟,
클래스, y

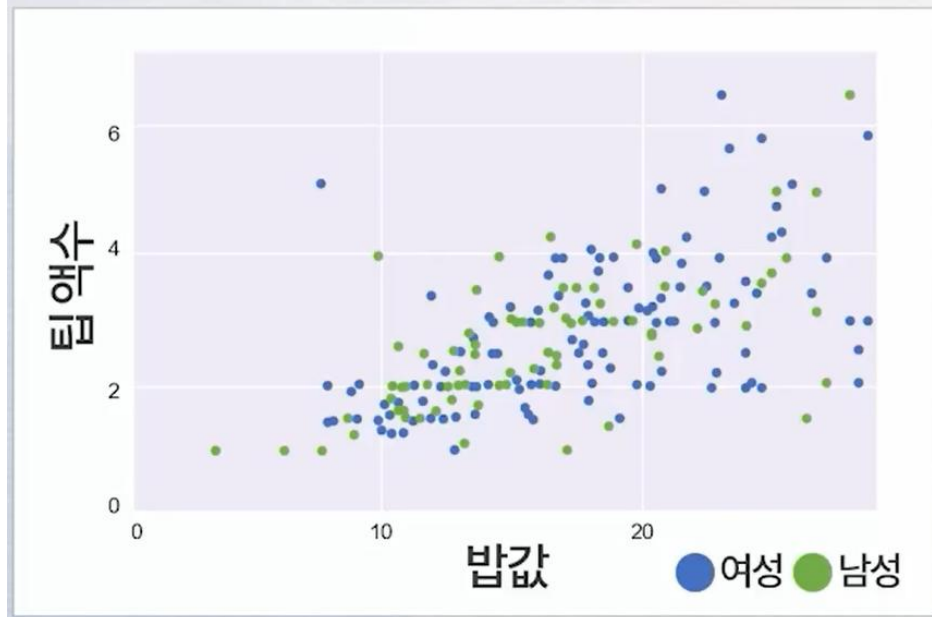
예측된 값 = 예측값, 분류값, $\hat{y}$ (y hat)

VS

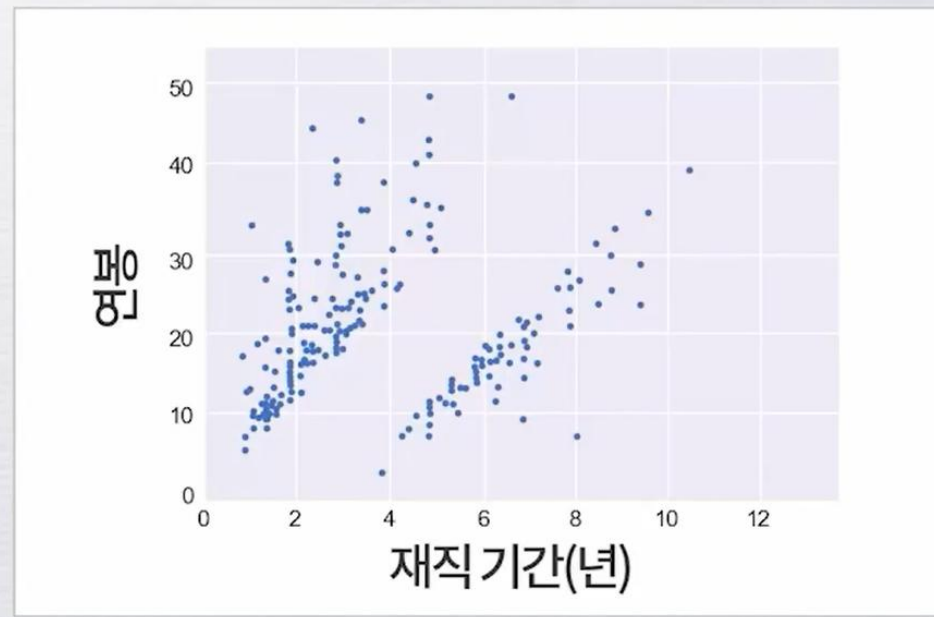
비지도학습 (Un-Supervised Learning)

- ▶ 레이블(정답) 없이 진행되는 학습
- ▶ 데이터 자체에서 패턴을 찾아내야 할 때
사용함

지도학습 vs. 비지도학습



분류모델



군집분석

지도학습 모델 종류

분류모델 (Classification)

레이블의 값들이 **이산적으로**
나뉘질 수 있는 문제에 사용

Example

밥값이 많이 나오면 남성,
적게 나오면 여성

예측 모델 (Regression, 회귀모델)

레이블의 값들이 **연속적인**
문제에 사용

Example

밥값이 많이 나올수록,
팁의 크기도 계속 커짐

지도학습 모델 종류

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

예측 모델(회귀 모델) 
팁의 크기를 예측

타겟변수를
tip으로 설정하는 경우

분류 모델 
손님의 성별을 예측

타겟변수를
sex로 설정하는 경우

지도학습 데이터셋 구조

각 열(column)을 **특징/속성(feature)** 이라고 함

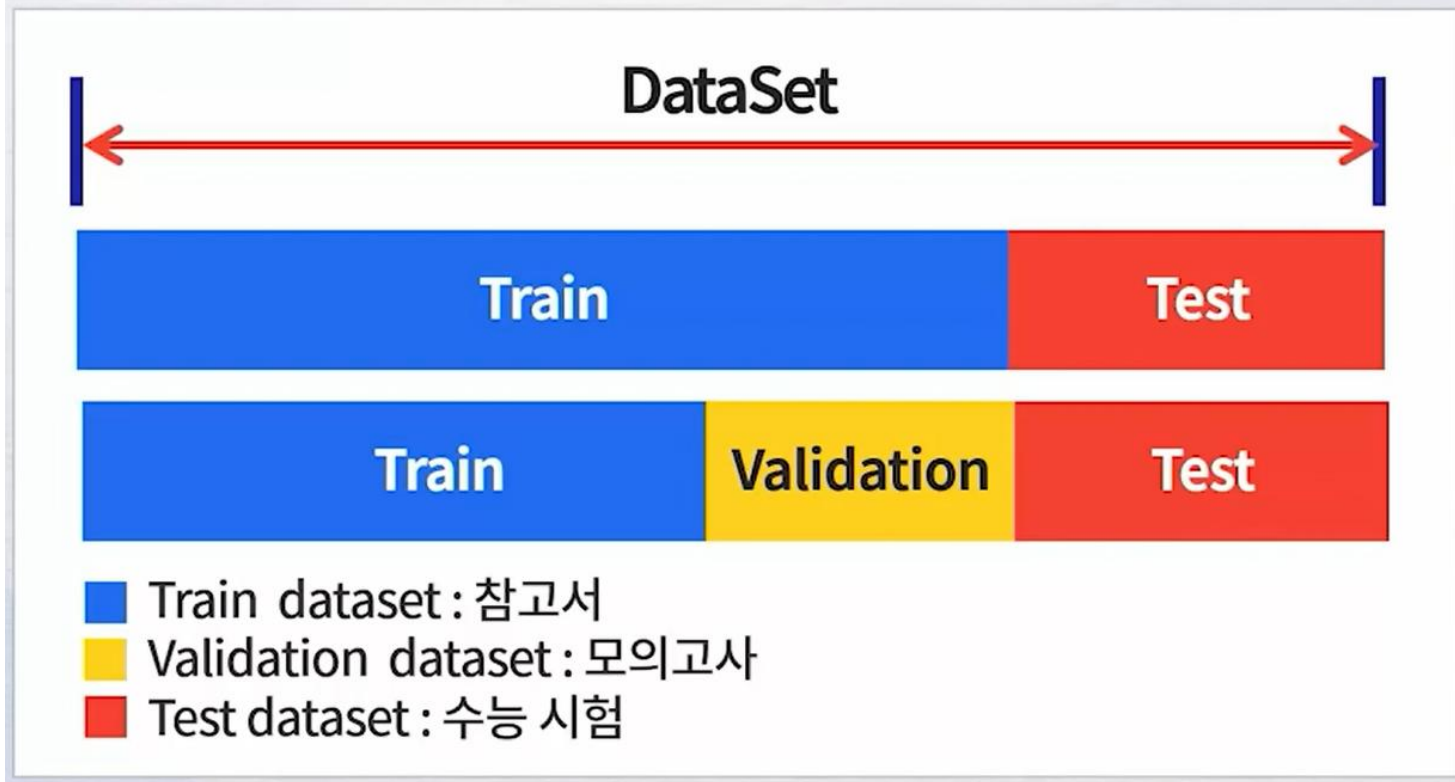
각 행(row)을 **예제(Example)** 데이터라고 함

	x_1	y	x_2	x_3	x_4	x_5	x_6
	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

데이터 열(column) 중에 하나를 선택해서 **레이블**로 사용함

$$y = wx$$

데이터셋 분리



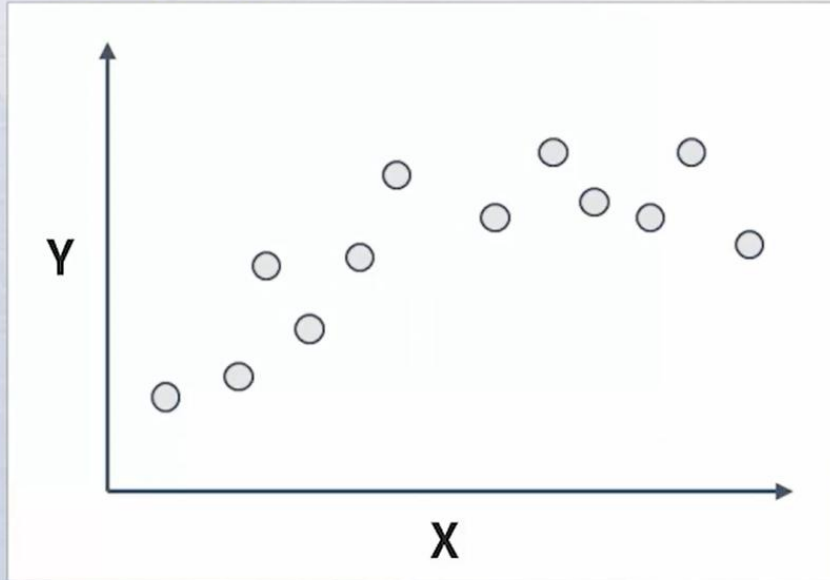
7:3

8:2

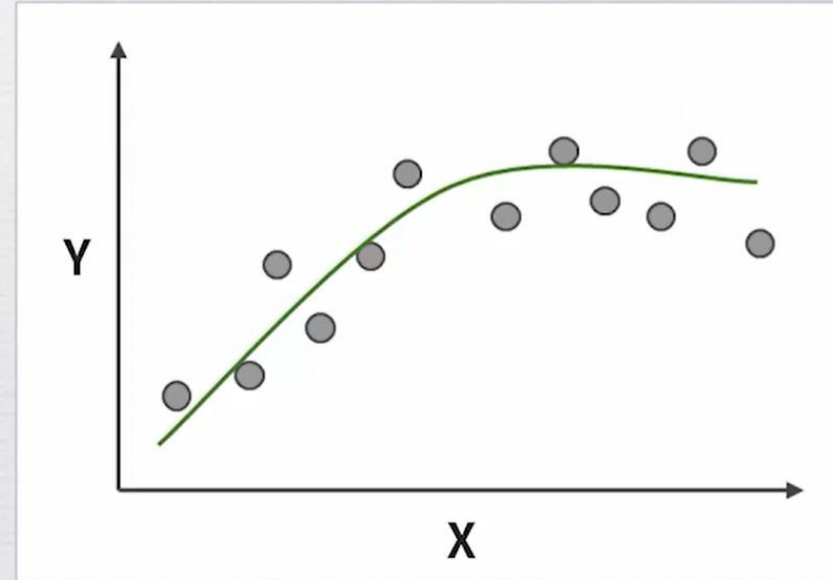
모델 선택



데이터



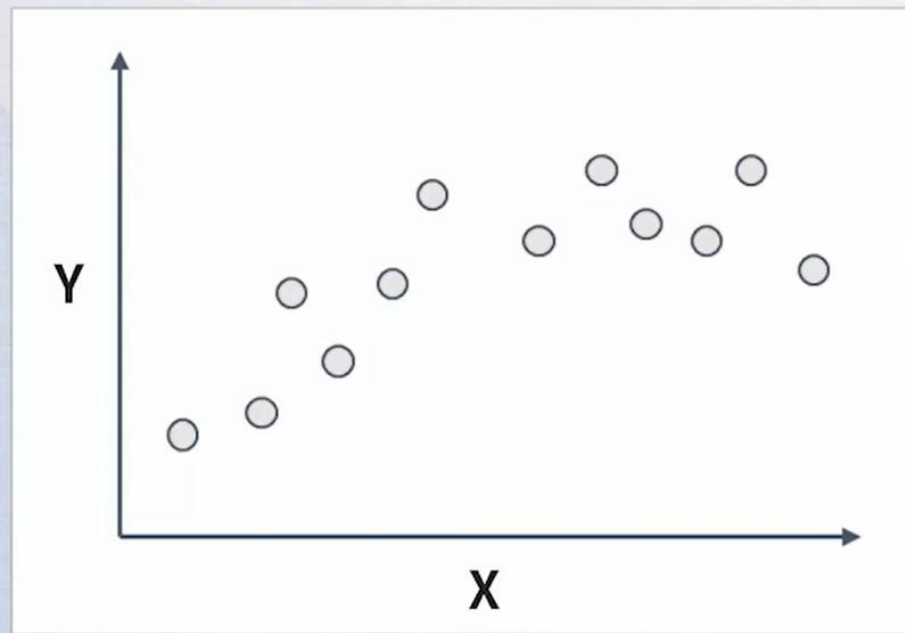
좋은 모델(Model)



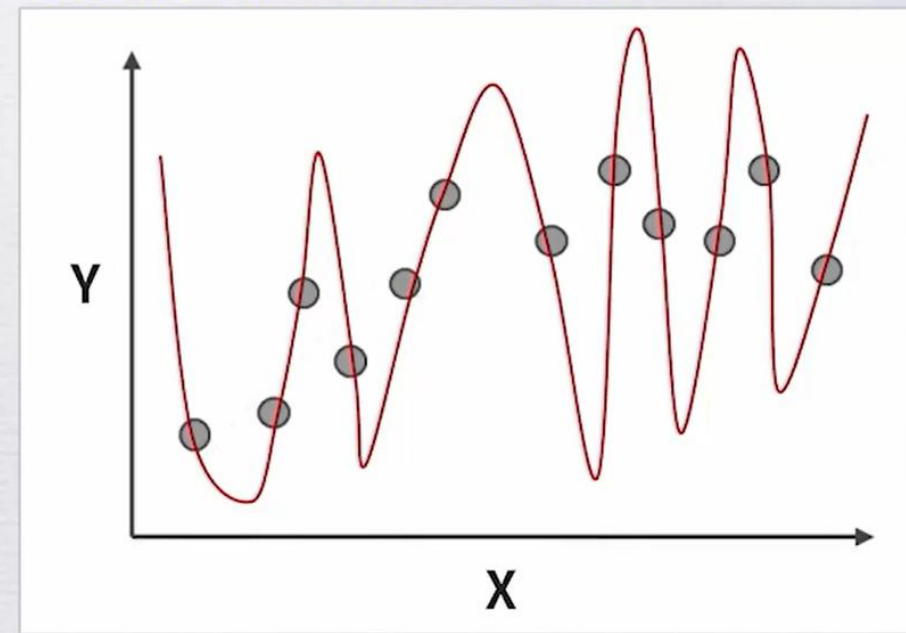
모델 선택



데이터



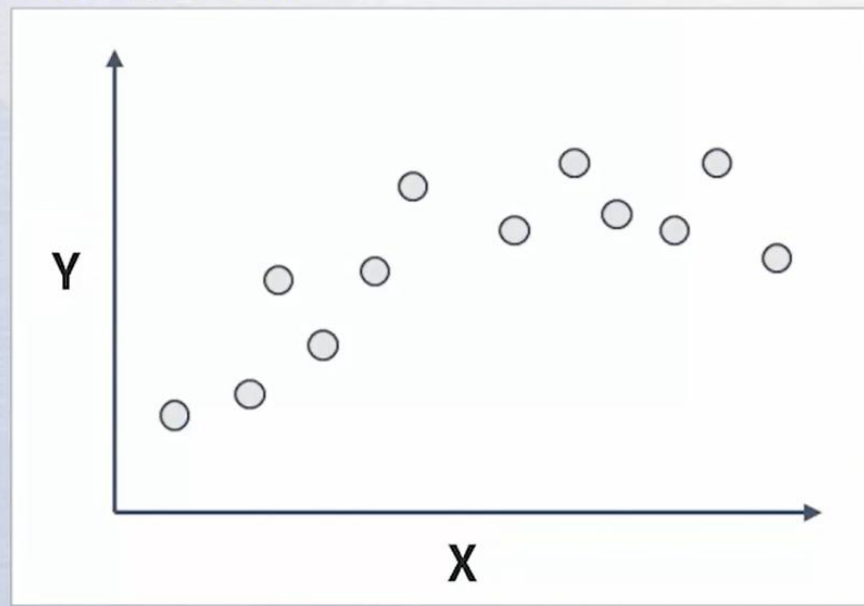
과적합(Overfitting)



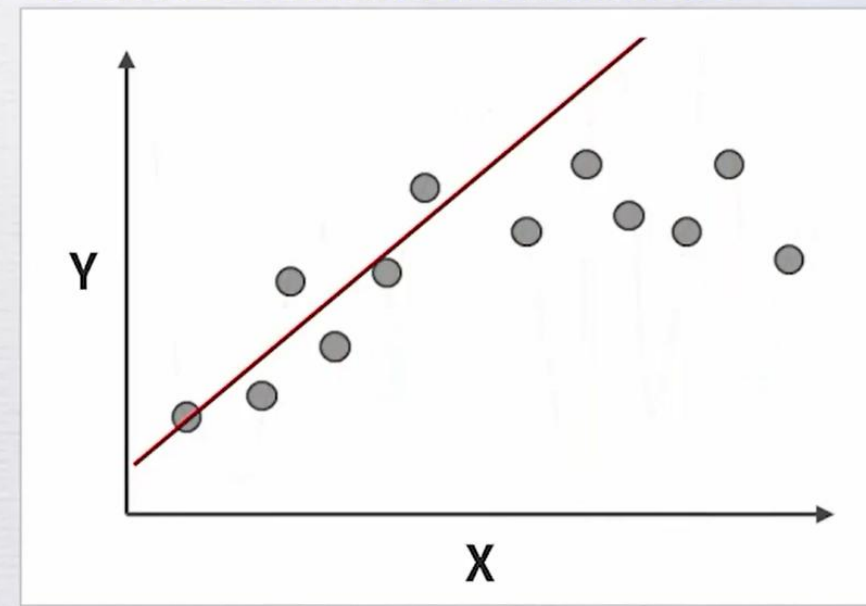
모델 선택



데이터

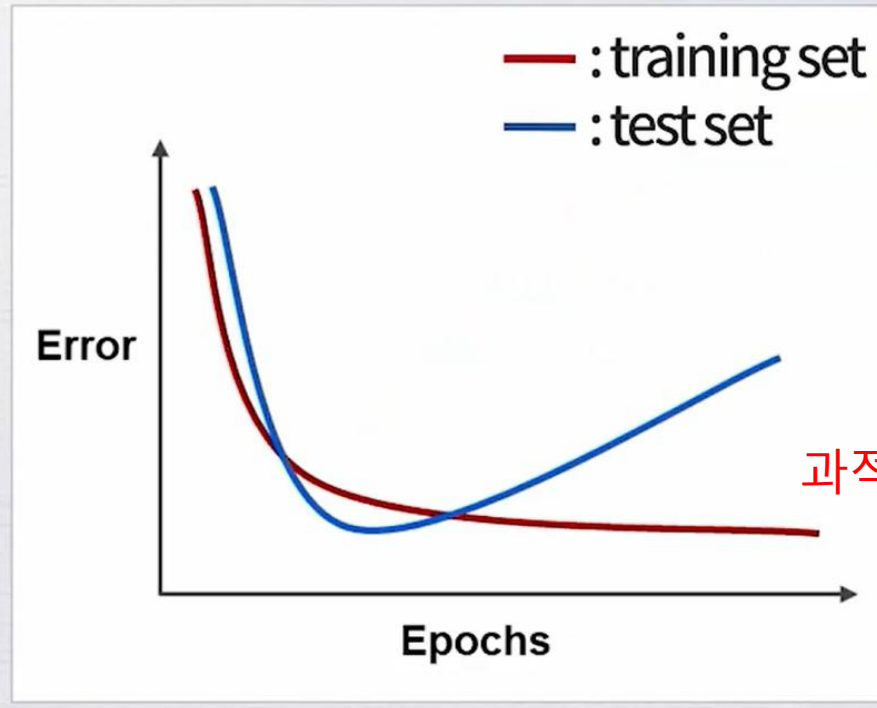
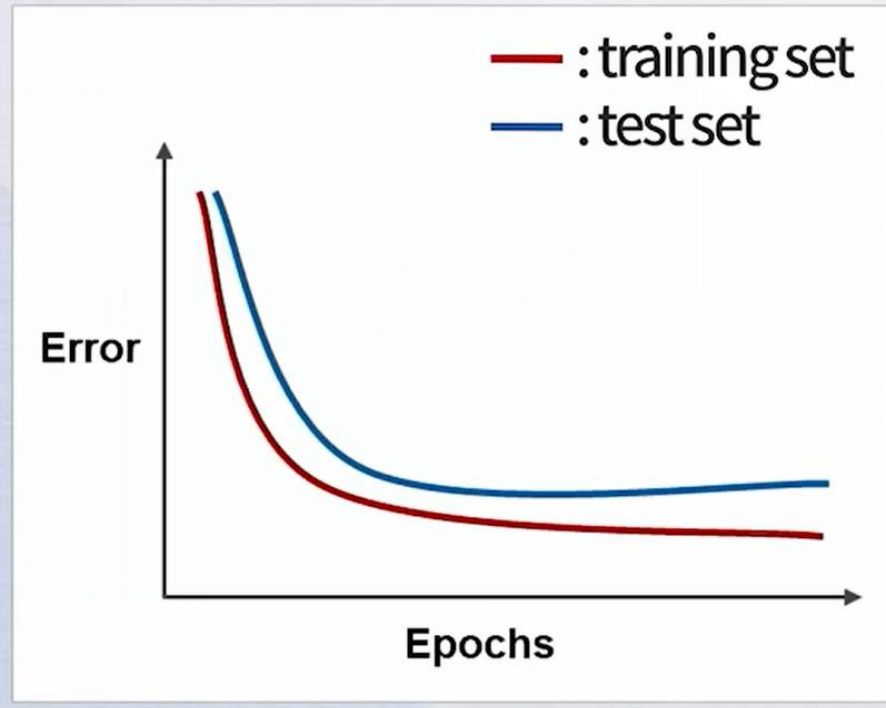


과소적합(Underfitting)



모델 성능 평가

$\text{error} = (\text{실제값} - \text{예측값})$
Epochs = 학습횟수



Confusion Matrix



모델의 대략적인 성능확인과 **모델의 성능**을
오차행렬을 기반으로 수치로 표현

Confusion Matrix

		예측값	
		0	1
실제값	0	TN	FP
	1	FN	TP

- ▶ True Positive(TP): True를 True로 예측 (정답)
- ▶ True Negative(TN): False를 False로 예측 (정답)
- ▶ False Positive(FP): False를 True로 예측 (오답)
- ▶ False Negative(FN): True를 False로 예측 (오답)

맞느냐 틀렸느냐 → **T N** ← 예측한 값

Confusion Matrix
분류모델에 대한 성능지표 계산을
위한 데이터

성능지표



학습이 끝난 후 **모델을 평가**하는 용도로 사용됨

Confusion Matrix

		예측값	
		0	1
실제값	0	TN	FP
	1	FN	TP

성능지표

암환자 진단은
재현율 중요

정밀도(Precision)

- ▶ 모델이 **True**라고 분류한 것 중에서 **실제 True**인 것의 비율
- ▶ 날씨 예측 모델이 맑다로 예측했는데, 실제 날씨가 맑았는지 나타낸 지표

$$(Precision) = \frac{TP}{TP + FP}$$

재현율(Recall)

- ▶ **실제 True**인 것 중에서 모델이 **True**라고 예측한 것의 비율
- ▶ 실제 날씨가 맑은 날 중에서 모델이 맑다고 예측한 비율을 나타낸 지표

$$(Recall) = \frac{TP}{TP + FN}$$

Confusion Matrix

		예측값	
		0	1
실제값	0	TN	FP
	1	FN	TP

정확도(Accuracy)

- ▶ 가장 직관적으로 모델의 성능을 나타낼 수 있는 평가 지표
- ▶ 한 달에 맑은 날이 28일이고 비가 오는 날이 2일인 경우, 비가 오는 것을 예측하는 성능은 매우 낮을 수 밖에 없으므로 이를 보완할 지표가 필요

$$(Accuracy) = \frac{TP + TN}{TP + FN + FP + TN}$$

성능지표

F1 점수(F1-score)

▶ 정밀도와 재현율의 조화평균

$$(F1-score) = 2 \times \frac{1}{\frac{1}{Precision} + \frac{1}{Recall}}$$

예제

Confusion Matrix

		예측값	
		0	1
실제값	0	2	0
	1	2	2

정밀도: $2/2=1$

재현율: $2/4=0.5$

실제값: $y_true = [1, 0, 1, 1, 0, 1]$

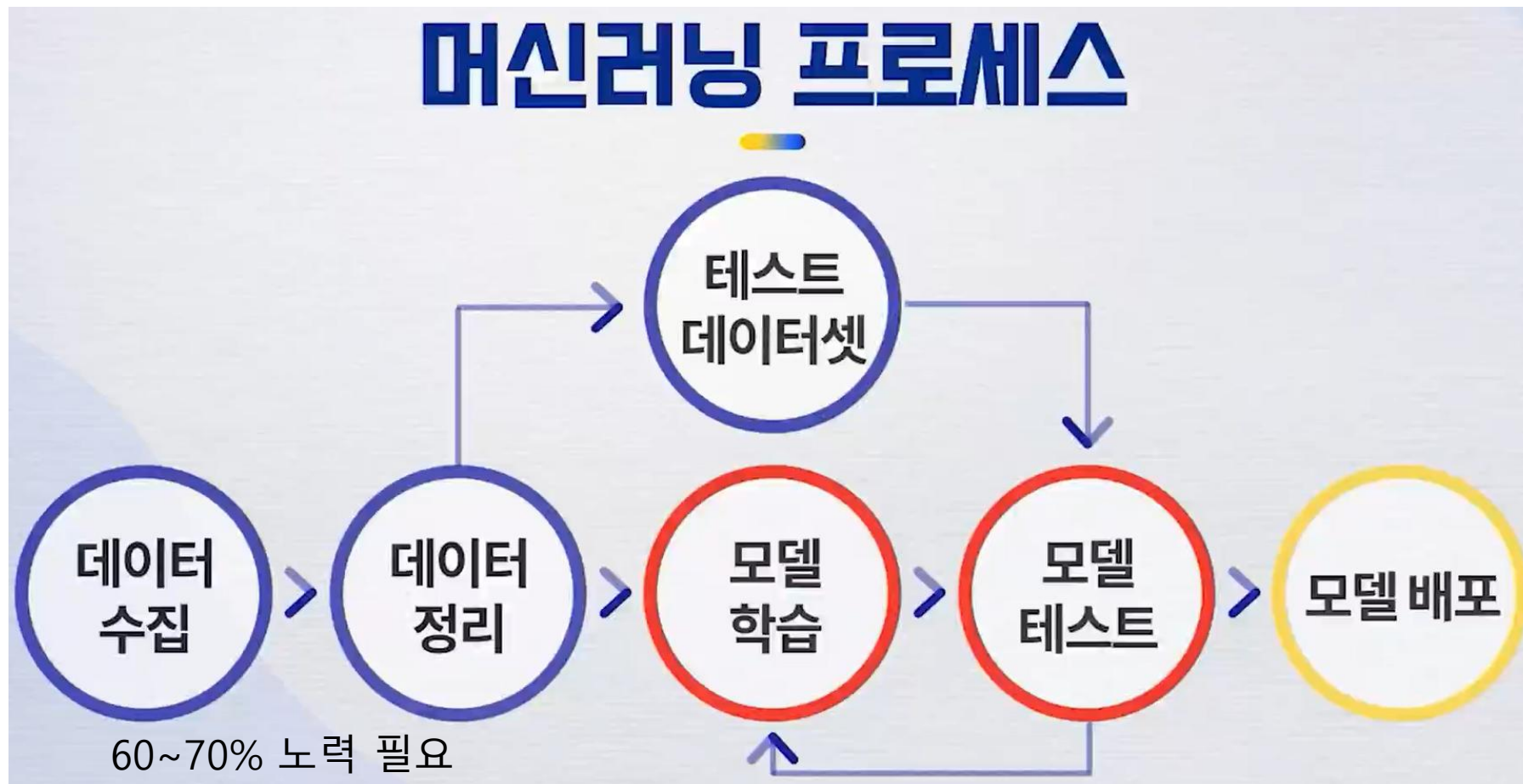
예측값: $y_pred = [0, 0, 1, 1, 0, 0]$

정밀도

$$(Precision) = \frac{TP}{TP + FP}$$

재현율

$$(Recall) = \frac{TP}{TP + FN}$$



머신러닝의 주요 알고리즘(1/27)

사이킷런 라이브러리

The Scikit-learn logo, featuring the text "Scikit-learn" in a bold, blue, sans-serif font. Below the text is a small, horizontal, multi-colored bar (yellow, orange, red, blue) that serves as a decorative element.

가장 인기 있는 머신러닝 패키지,
많은 머신러닝 알고리즘이 내장되어 있음

파이썬 코딩에서 머신러닝 기반 학습시킬 때 필요

Scikit-learn

```
from sklearn.family import Model
```



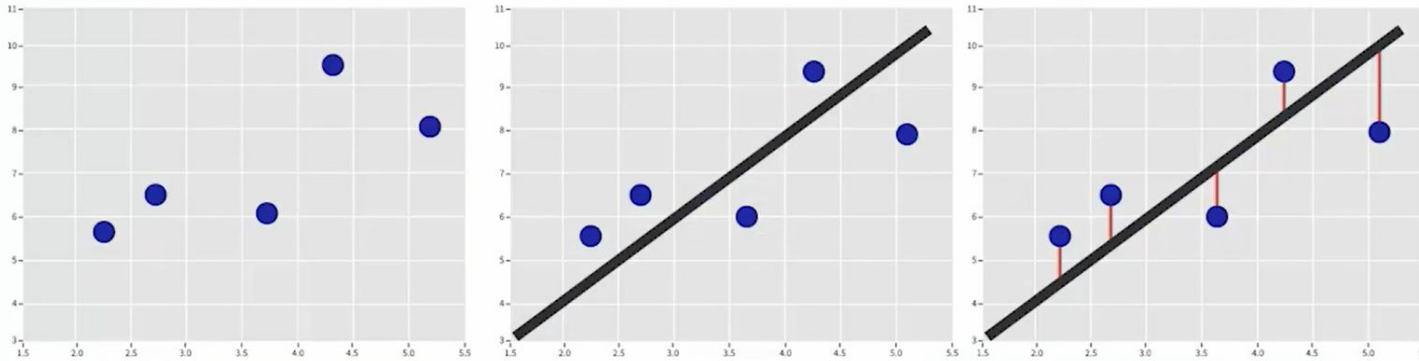
```
from sklearn.linear_model import LinearRegression  
model = LinearRegression()  
print(model)
```



Linear Regression

데이터의 분포 패턴에 가장 적합한 직선을 찾아서 예측

$$y = wx$$



Linear Regression

```
from sklearn.linear_model import LinearRegression
```

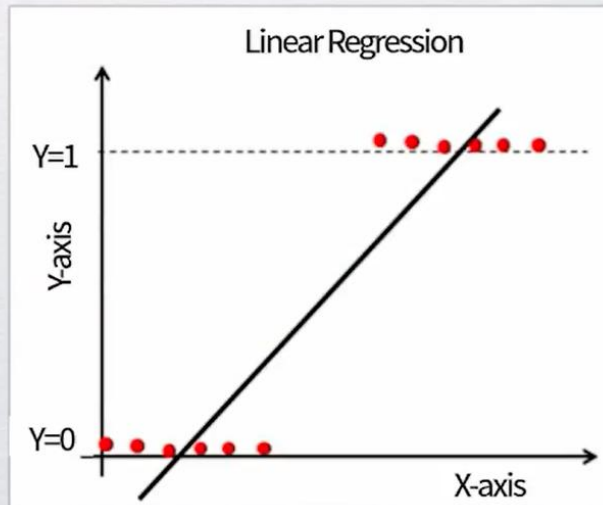
```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

```
pred = model.predict(X_test)
```

Logistic Regression

이진 분류 규칙은 0과 1의 두 클래스를 갖는 것으로,
일반 선형 회귀 모델을 이진분류에 사용하기 어려움



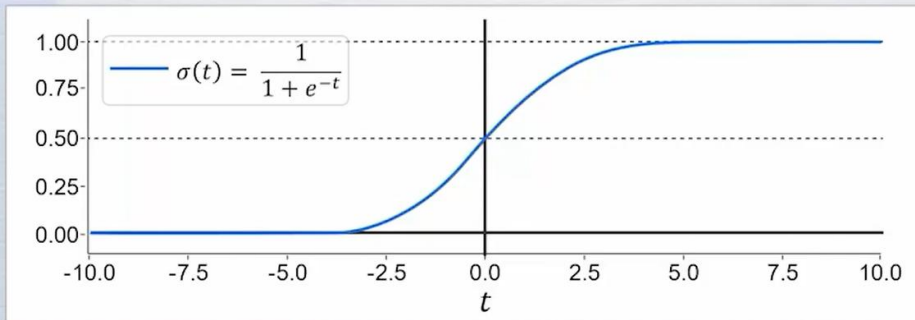
$$y = wx$$

출처: <https://ichi.pro/ko/lojiseutig-hoegwi-ihae-mich-python-eseo-model-guchug-25643771585192>

Logistic Regression

로지스틱(logistic) 함수를 사용하여 로지스틱 회귀 곡선으로
변환하여 이진 분류 할 수 있음

✓ 로지스틱 함수



$$y = wx$$
$$\hat{y} = \begin{cases} 0 & \text{if } y < 0.5 \\ 1 & \text{if } y \geq 0.5 \end{cases}$$

시그모이드(=로지스틱) 함수:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- 입력 z 를 0~1 사이의 확률 값으로 변환
- S자(S-shape) 곡선을 가짐

$$y = wx$$

$$\hat{y} = \frac{1}{1 + e^{-wx}}$$

Logistic Regression

```
from sklearn.linear_model import LogisticRegression
```

```
model = LogisticRegression()
```

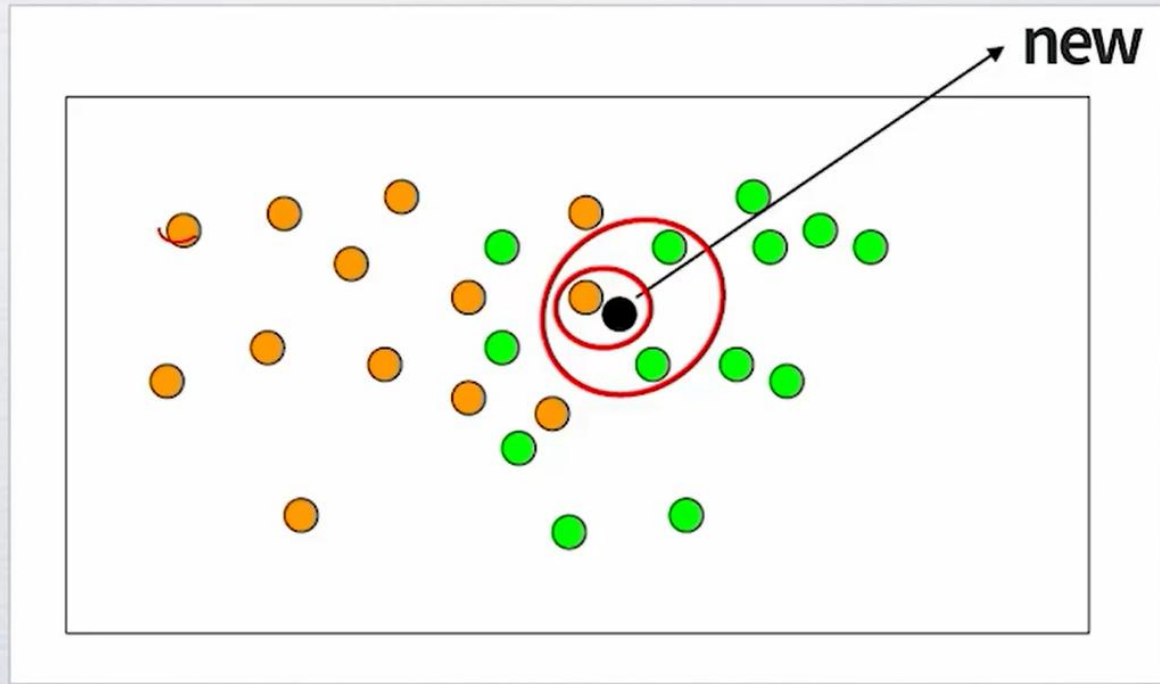
```
model.fit(X_train, y_train)
```

```
pred = model.predict(X_test)
```


K-Nearest Neighbor

새로운 데이터가 주어졌을 때 기존 데이터 가운데
가장 가까운 k 개 이웃의 정보로 새로운 데이터를 예측하는 방법론

K-Nearest Neighbor



k 파라미터
: 이웃의 갯수

k = 1 : 검은점은 노란색
k = 3 : 검은 점은 연두색

K-Nearest Neighbor

알고리즘이 간단하며
큰 데이터셋과 고차원 데이터에 적합하지 않음

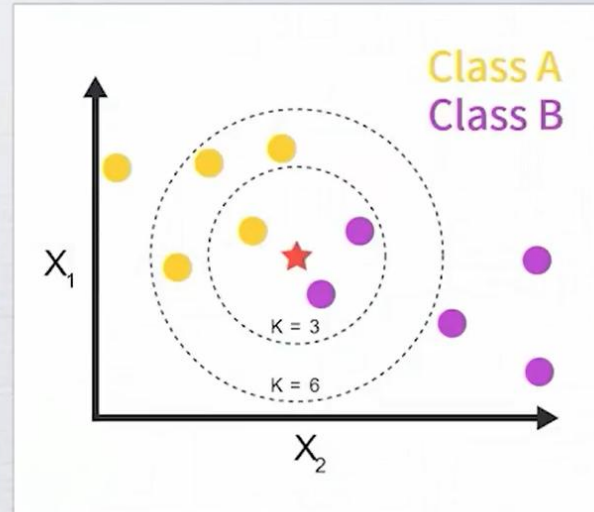
K-Nearest Neighbor

```
from sklearn.neighbors import KNeighborsClassifier
```

```
knn = KNeighborsClassifier(n_neighbors=3)
```

```
knn.fit(X_train, y_train)
```

```
pred = knn.predict(X_test)
```



Decision Tree

분류와 회귀 작업이 가능한 다재다능한 머신러닝 알고리즘



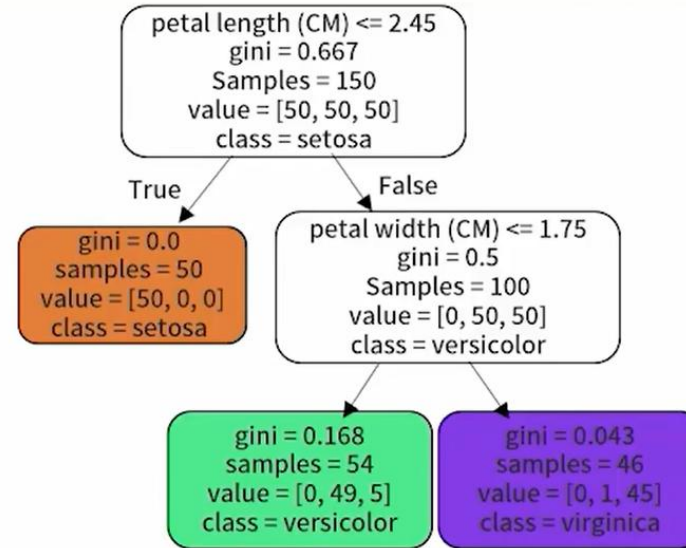
복잡한 데이터셋도 학습할 수 있으며
강력한 머신러닝 알고리즘인 **랜덤 포레스트**의 기본 구성 요소

Decision Tree

✓ 붓꽃(Iris)



✓ 붓꽃 분류 결정트리



Decision Tree

```
from sklearn.tree import DecisionTreeClassifier
```

```
model = DecisionTreeClassifier(max_depth=2)
```

```
model.fit(X_train, y_train)
```

```
pred = model.predict(X_test)
```

Random Forest

일련의 예측기(분류, 회귀모델)로부터 예측을 수집하면
가장 좋은 모델 하나보다 더 좋은 예측을 얻을 수 있음

일련의 예측기

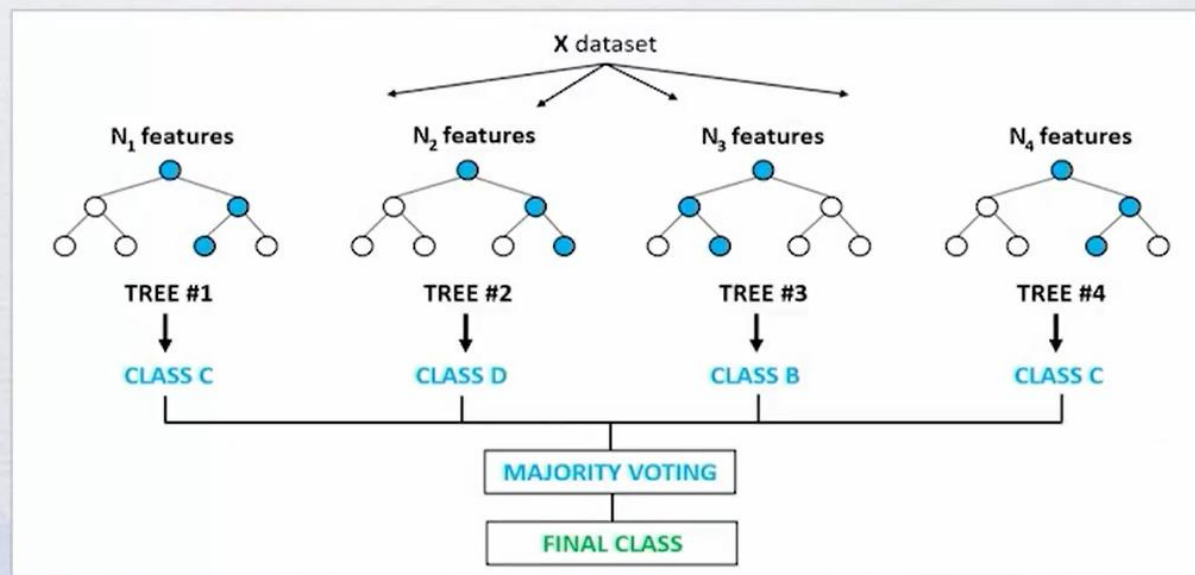
결정 트리의 앙상블

앙상블

랜덤
포레스트

Random Forest

훈련 세트로부터 무작위로 각기 **다른 서브셋**을 만들어
일련의 결정 트리 분류기를 훈련시킬 수 있음



출처:
<https://www.kaggle.com/getting-started/176257>

Random Forest

```
from sklearn.ensemble import RandomForestClassifier
```

```
model = RandomForestClassifier(n_estimators=50)
```

```
model.fit(X_train, y_train)
```

```
pred = model.predict(X_test)
```


Ensemble

여러 개의 분류기를 생성하고, 그 **예측을 결합**함으로써
보다 정확한 예측을 도출하는 기법



약한 모델 여러 개를 조합하여 더 정확한 예측 방식
적절한 Hyperparameter 튜닝이 중요

Ensemble 기법

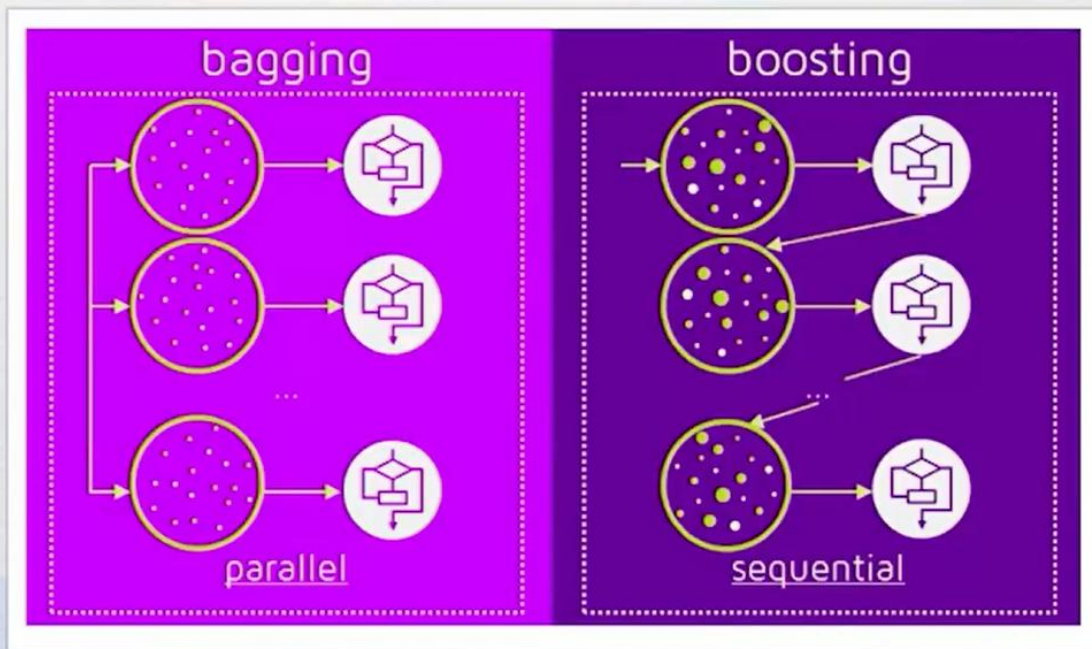
1) Boosting

- ▶ 이전 학습에 대하여 잘못 예측된 데이터에 가중치를 부여해 오차를 보완해 나가는 방식
- ▶ 대표 모델 : XGBoost, LightGBM

**순차적인 학습을 하며 weight를 부여해서
오차를 보완하므로 학습 시간이 길수 있음**

Ensemble 기법

1) Boosting



출처:

<https://medium.com/swlh/difference-between-bagging-and-boosting-f996253acd22>

대표적인 bagging 알고리즘
: RandomForest

대표적인 boosting 알고리즘
: XGboost, LightGBM

Ensemble 7법

2) Stacking

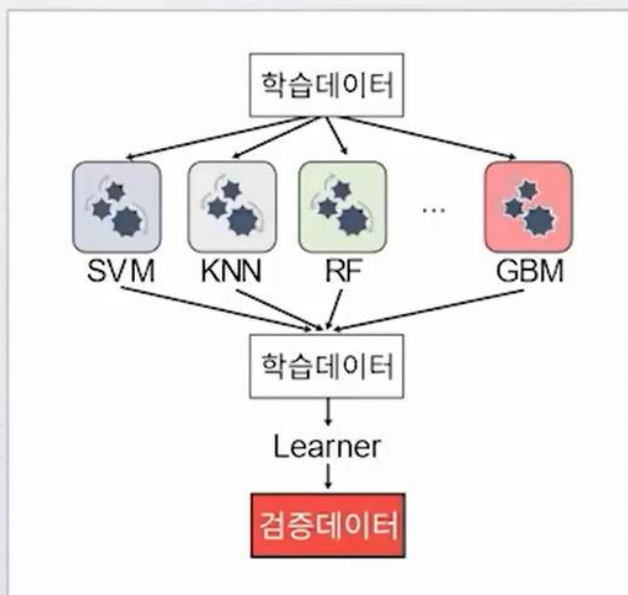
- ▶ 여러 개 모델이 예측한 결과 데이터를 기반으로 `final_estimator` 모델로 종합하여 예측 수행



성능은 향상될 수 있으나 과대적합을 유발할 수 있음

Ensemble 7법

2) Stacking



출처 : <https://pangyo-datascientist.tistory.com/entry/%EC%95%99%EC%83%81%EB%B8%94-Stacking-ensemble-%EC%9D%98-ensemble>

Ensemble 기법

3) Weighted Blending

▶ 각 모델의 예측값에 대하여 weight를 곱하여 최종 output 계산



가중치의 합은 **1.0**이 되도록 함

KNN(가중치 0.25)

RandomForest(가중치 0.5)

Logistic Regression(가중치 0.25)

Ensemble 대표 모델

1) XGBoost

```
!pip install xgboost  
from xgboost import XGBClassifier
```

```
model = XGBClassifier(n_estimators=50)
```

```
model.fit(X_train, y_train)
```

```
pred = model.predict(X_test)
```

Ensemble 대표 모델

2) LightGBM

```
!pip install lightgbm  
from xgboost import LGBMClassifier
```

```
model = LGBMClassifier(n_estimators=50)
```

```
model.fit(X_train, y_train)
```

```
pred = model.predict(X_test)
```

머신러닝의 주요 알고리즘(27/27)

기본 개념

- ▶ 최적의 선을 긋는 것
- ▶ Cost Function
- ▶ Gradient Decent
(경사하강법)

기술 원리

- ▶ 지도학습 vs 비지도학습
- ▶ 회귀와 분류
- ▶ 데이터 확보/전처리 시간
소요
- ▶ 모델 성능 지표

주요 알고리즘

Linear Regression

Logistic Regression

Decision Tree

LightGBM

XGBoost

Random Forest

파이썬 머신러닝 실습

```
(4)-머신러닝.py x
1  """
2  #####
3  [실습-정답] Python을 활용한 AI 모델링 - 머신러닝 파트
4  #####
5
6  - 이번시간에는 Python을 활용한 AI 모델링에서 머신러닝에 대해 실습해 보겠습니다.
7  - 머신러닝 모델에는 아래와 같이 모델들이 있습니다.
8      => 단일 분류예측 모델 : LogisticRegression, KNN, DecisionTree
9      => 앙상블(Ensemble) 모델 : RandomForest, XGBoost, LGBM, Stacking, Weighted Blending
10 - 솔직히, 머신러닝이 딥러닝보다 코딩하기 쉽습니다. 4줄 템플릿에 맞춰 코딩하면 되기 때문입니다.
11 - 한가지 당부 드리고 싶은 말은 "백문이불여일타" 입니다.
12 - 이보다 실습이 더 많은 시간과 노력이 투자 되어야 합니다.
13
14 #####
15 학습목차
16 #####
17
18
19 1. 머신러닝 모델 프로세스
20 - 데이터 가져오기
21 - 데이터 전처리
22 - Train, Test 데이터셋 분할
23 - 데이터 정규화
24 - 단일 분류예측 모델 : LogisticRegression, KNN, DecisionTree
25 - 앙상블(Ensemble) 모델 : RandomForest, XGBoost, LGBM, Stacking, Weighted Blending
26
27 """
28
29 ## 1. 실습준비
30 """
31 # 코드실행시 경고 메시지 무시
32
33 import warnings
34 warnings.filterwarnings(action='ignore')
35 """
36
37 """
38 #####
39 2. 머신러닝 모델 프로세스
40 #####
```


감사합니다.